

웹 취약점 진단 보고서

Web Application Vulnerability Assessment Report

1. 진단 사이트 개요

진단 호스트	localhost / Target Server (Masked)
종합 진단 결과	취약 발견 (VULNERABLE)

본 보고서는 마스킹 처리된 패킷 데이터를 기반으로 LLM이 진단한 결과 및 N/A 판정 근거를 포함하며, 개발 언어 미특정 환경을 고려하여 주요 프레임워크별 시큐어코딩 가이드를 제공합니다.

2. 진단 결과 요약 및 상세 분석

취약점 명칭	Cross-Site Scripting (XSS)
OWASP 분류	A05 Injection
위험도 (Severity)	MEDIUM
판단 근거 요약	입력된 'zipcode' 값이 HTML 컨텍스트에 반영되고 있으며, 이 값이 적절히 인코딩되지 않으면 XSS 공격에 취약할 수 있습니다.
요청 분석 근거 (Request)	• Body에 'zipcode' 파라미터가 포함되어 있으며, 이는 사용자 입력을 통해 조작될 수 있습니다.
응답 분석 근거 (Response)	• HTML 응답에서 'zipcode' 입력값이 그대로 반영되고 있으며, 인코딩이나 필터링 없이 출력될 가능성이 있습니다.
보안 영향 (Impact)	공격자가 악성 스크립트를 삽입하여 사용자의 세션 탈취, 웹사이트 변조, 피싱 공격 등을 수행할 수 있습니다.
추가 확인사항 (N/A 전용)	None

3. 프레임워크별 시큐어코딩 및 패치 가이드

대응 방안 요약: 사용자 입력을 HTML 컨텍스트에 반영하기 전에 적절히 인코딩하거나 필터링하여 XSS 공격을 방지하십시오.

Spring (Java)

취약한 코드 (Vulnerable)

```
// 사용자 입력을 그대로 모델에 담아 뷰에 전달
model.addAttribute("input", userInput);
```

안전한 패치 코드 (Secure)

```
// Spring Security 또는 OWASP Encoder를 통한 HTML Entity Encoding 적용
```

Flask (Python)

취약한 코드 (Vulnerable)

```
# Jinja2 템플릿에서 |safe 필터 사용 또는 안전하지 않은 렌더링
return render_template_string(f"<div>{user_input}</div>")
```

안전한 패치 코드 (Secure)

```
# Jinja2는 기본적으로 자동 이스케이핑(Auto-escaping) 지원
return render_template("index.html", user_input=user_input)
```

Node.js

취약한 코드 (Vulnerable)

```
res.send(`<div>${userInput}</div>`);
```

안전한 패치 코드 (Secure)

```
// DOMPurify 또는 escape-html 라이브러리 사용
res.send(`<div>${escapeHtml(userInput)}</div>`);
```